

Bridge Design Pattern

• Introduction:

- The **Bridge Design Pattern** is a **structural** design pattern that separates:
 - **Abstraction** → High-level business logic
 - **Implementation** → Low-level details
- So that both can vary independently.

• Problem Statement:

We need to model:

- Car Types
 - SUV
 - Sedan
 - Hatchback
- Engine Types
 - Petrol
 - Diesel
 - Electric
- Each car can have any type of engine.

• The Problem: Class Explosion

What is **Class Explosion**?

- **Class explosion** happens when multiple dimensions of variation are modelled using inheritance, leading to a combinatorial increase in the number of classes.
- If:
 - A system has **M types** in one dimension
 - And **N types** in another dimension
- Total classes required = **M × N**
- As dimensions grow, the number of classes grows exponentially.

In this scenario if we try to model combinations using **inheritance**:

1. SUV_Petrol
2. SUV_Diesel
3. SUV_Electric
4. Sedan_Petrol
5. Sedan_Diesel
6. Sedan_Electric
7. Hatchback_Petrol
8. Hatchback_Diesel
9. Hatchback_Electric

Calculation

- 3 Car types
- 3 Engine types

Total classes = $3 \times 3 = 9$ classes

Bridge Design Pattern

• The Problem: Scaling

- If later we add:
 - New Engine → Hybrid
 - New Car → Coupe
- Now:
 - 4 Car types
 - 4 Engine types
- Total classes = $4 \times 4 = 16$ classes
- If we add more variations (e.g., transmission type), complexity becomes unmanageable.

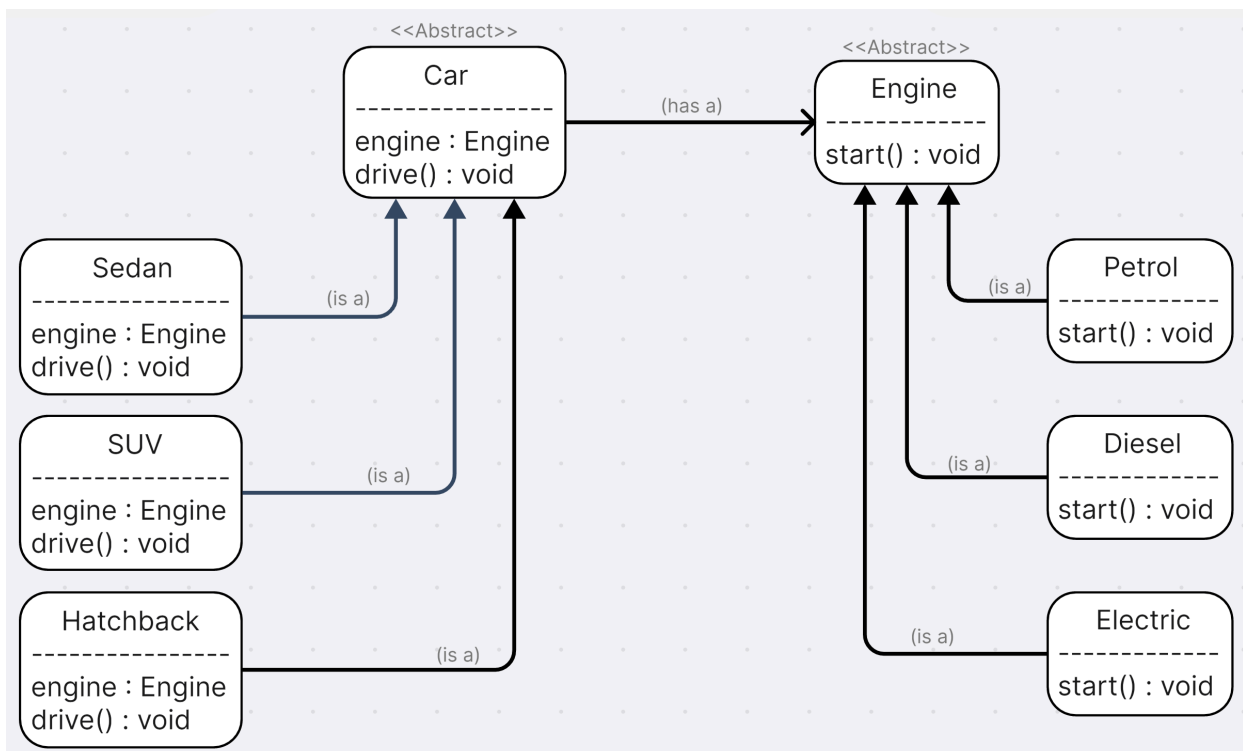
• Problems:

- High maintenance cost
- Tight coupling between Car and Engine
- Difficult to extend
- Violates Open-Closed Principle
- Difficult to test and refactor

• Bridge Pattern – The Solution

- Instead of combining both dimensions using inheritance:
- We separate:
 - Abstraction → Car
 - Implementation → Engine
- And connect them using **composition**.
- Car HAS-A Engine
- Instead of Car IS-A Engine combination.

• Structure of Bridge Pattern:



Bridge Design Pattern

- Now:
 - Adding new Car type does NOT affect Engine classes
 - Adding new Engine type does NOT affect Car classes
- Total classes = $M + N$
Not $M \times N$
- For our case:
 - 3 Cars
 - 3 Engines
- Total classes = 6 (instead of 9)

• Code Link:

https://github.com/sibasundarj8/-System-Design--LLD-/tree/main/Codes/25__Bridge%20Pattern%20code

• How Bridge Solves Class Explosion:

Without Bridge	With Bridge
Uses inheritance	Uses composition
Classes = $M \times N$	Classes = $M + N$
Tight coupling	Loose coupling
Hard to extend	Easy to extend
Violates OCP	Follows OCP

• Design Principles Applied:

- Composition over Inheritance
- Open-Closed Principle
- Dependency Inversion Principle
- Separation of Concerns

• Conclusion:

- The **class explosion problem** occurs when multiple independent dimensions are modelled using inheritance, causing exponential growth in classes.
- The **Bridge Pattern** solves this by:
 - Separating abstraction from implementation
 - Using composition instead of inheritance
 - Allowing independent extensibility
 - Reducing class count from $M \times N$ to $M + N$